Brawl Stars Ranked Draft AI

Projektspezifikation & Build-Prompt für Claude Code · Ausgangsbasis: bestehendes Repo, siehe Abschnitt 2

Dieses Dokument ist der vollständige Kontext für den Weiterbau eines persönlichen Projekts von Kian (Informatik-Student, IMS Zürich / BZZ, Fachrichtung Applikationsentwicklung): ein Web-Tool, das während eines echten Brawl Stars **Ranked**-Drafts (Zielrangbereich Legendary III bis Pro) live den statistisch besten nächsten Ban oder Pick vorschlägt – trainiert auf echten, über die offizielle API gesammelten Matchdaten. Nutze dieses Dokument als vollständigen Projektkontext, nicht als starre Schritt-für-Schritt-Anleitung. Wo etwas noch unverifiziert oder eine offene Designentscheidung ist, ist das explizit markiert – dort bitte verifizieren bzw. sinnvoll entscheiden statt zu raten oder unbegründet anzunehmen.

---

0. Tech-Stack (verbindlich, nicht offen)

Damit nichts geraten werden muss: der Stack ist wie folgt festgelegt.

- **Backend:** Python 3.11+, FastAPI, Uvicorn. Modell (Abschnitt 8) wird per `joblib`/pickle persistiert und beim Start einmal geladen (siehe Abschnitt 10).
- **Frontend:** einfaches HTML/CSS/Vanilla-JS, **kein** Framework, **kein** Build-Step. Wird von derselben FastAPI-App als statische Datei ausgeliefert (`StaticFiles`-Mount) – ein einziger Prozess/Service für Backend und Frontend zusammen, kein separates Node-Deployment.
- **Datenbank:** SQLite (bereits implementiert), auf einem persistenten Railway-Volume.
- **Begründung:** nur ein Railway-Service für Backend+Frontend ist einfacher und günstiger als zwei getrennte Services, und das UI selbst (Map wählen, Brawler anklicken, Empfehlung anzeigen) ist nicht komplex genug, um eine Framework-/Build-Pipeline zu rechtfertigen. Falls bewusst Framework-Erfahrung gewünscht ist: React + Vite ist die Alternative, zu statischen Assets gebaut und weiterhin von FastAPI ausgeliefert (bleibt bei einem Service).

---

1. Zielsetzung

Primäres Ziel: ein interaktives Web-Tool, das während eines laufenden Ranked-Drafts – gegeben Map, bisherige Bans beider Teams und bisherige Picks beider Teams – die beste nächste Aktion vorschlägt. Die Empfehlung darf sich **nicht** auf simples Sortieren nach globaler Winrate beschränken. Zwei Anforderungen sind zentral und stammen direkt aus der Projektdiskussion:

a) Denial beim Bannen

Ein guter Ban ist nicht zwingend der Brawler mit der höchsten globalen Winrate, sondern der, dessen Entfernung dem Gegner das grösstmögliche verbleibende Potenzial nimmt. Beispiel aus der Projektdiskussion: bei First Pick sollen als beste Bans die Brawler vorgeschlagen werden, die die höchste Winrate *gegen* die eigenen stärksten verfügbaren Picks haben – nicht einfach die generell stärksten Brawler.

b) Lookahead beim Picken

Ein Pick, der dem Gegner danach einen sehr starken Antwort-Pick ermöglicht, soll gemieden werden – zugunsten eines Picks, der die beste verfügbare gegnerische Antwort schwächt, auch wenn der eigene unmittelbare Pick isoliert betrachtet leicht schlechter aussieht.

Kurz: Das Draft-Modul ist eine **Minimax-/Spielbaum-Suche** über den verbleibenden Draft, keine einfache Sortierung nach Winrate. Details dazu in Abschnitt 9.

Referenz/Machbarkeitsnachweis: Es existieren bereits Community-Tools mit vergleichbarem Ansatz (z.B. „Deep Draft“ und „Draft Duel“, beide unter bourdin.alwaysdata.net) – Ban-Phase, alternierende Picks, Synergie-/Matchup-Matrizen aus gesammelten Matches, einstellbare Such-Engine-Tiefe. Nicht kopieren, aber als Beleg, dass der Ansatz grundsätzlich funktioniert, und optional als UX-Referenz fürs Frontend (Abschnitt 11) nützlich.

---

2. Bestehendes Repo (Ausgangspunkt – erweitern, nicht neu schreiben)

Folgende Module existieren bereits und implementieren die Abschnitte 4 und 7 dieses Dokuments:

- `src/api_client.py` – rate-limitierter Wrapper um die offizielle API, inkl. Backoff und 403-Behandlung bei IP-Wechsel des Keys.
- `src/db.py` – SQLite-Schema (players, battles inkl. rank-Spalte, battle_players), Dedup über Battle-Hash, Query-Helfer inkl. `high_rank_player_tags()`.
- `src/crawler.py` – Snowball-Crawler mit Priority-Heap (Priorität = rank des Kampfs, in dem ein Tag gefunden wurde), plus Reseed-Modus (`--reseed-from-rank`, `--recrawl-hours`) für periodisches Polling des bekannten High-Rank-Pools.
- `src/extract.py` – `inspect_types()`, `inspect_ranks()`, `build_training_table()` (inkl. Rang-Filter und -Gewichtung).
- `src/stats.py` – explorative Statistik: `brawler_winrate()`, `synergy_matrix()`, `matchup_matrix()`, mit Mindest-Stichprobenfilter.
- `README.md`, `requirements.txt`, `.env.example` – Setup und Roadmap.

Diese Dateien werden zusammen mit diesem Dokument bereitgestellt. Bitte darauf aufbauen.

---

3. Fakten über die offizielle API

Verifiziert:

- Kein Bulk-Endpoint. `battlelog(tag)` liefert nur die letzten ca. 25 Kämpfe eines Spielers über alle Modi gemischt, mit bis zu 30 Minuten Verzögerung für neue Einträge.
- Kein Draft-Log. Jeder Kampf-Eintrag enthält nur die finale 3v3-Zusammensetzung, Map, Ergebnis und `rank` (Rang des abgefragten Spielers zum Zeitpunkt des Kampfs) – keine Bans, keine Pick-Reihenfolge. Das ist eine harte Grenze der API, kein Bug im Crawler.
- API-Key ist an die öffentliche IP gebunden, mit der er erstellt wurde. Ändert sie sich, schlagen Requests mit 403 fehl.
- Keine eigene Ranked-Tier-Bestenliste. `/rankings/{country}/players` ist Trophäen-basiert (Trophy Road), nicht Ranked-Rang – nur eine korrelierte, unvollständige Näherung als Crawl-Seed.
- Aktuelle Rang-Reihenfolge (Stand des Ranked-Reworks 2026): Bronze, Silver, Gold, Diamond, Mythic, Legendary (I–III), Masters (I–III), Pro (neuester, höchster Rang oberhalb Masters, schaltet den Esports-Button frei).
- Ranked-Draft-Grundstruktur (aus Community-Draft-Tools bestätigt): 3 Bans pro Team (6 total), danach alternierende Einzel-Picks – kein Snake-Format mit Doppel-Picks.

Noch zu verifizieren (empirisch prüfen, nicht raten):

- Exakter `type`/`mode`-Stringwert für Ranked im Battlelog-JSON → `python -m src.extract types` nach dem ersten Crawl laufen lassen.
- Exakte numerische Kodierung von `rank` (welcher Wert entspricht Legendary III, welcher Pro) → `python -m src.extract ranks`, Ergebnis gegen Kians bekannten eigenen Rangverlauf abgleichen.
- Exakte Ban-Phasen-Reihenfolge im Detail (gleichzeitig oder sequenziell, welches Team beginnt). Öffentliche Quellen dazu sind dünn – am Live-Client beobachten/verifizieren, dann in `draft_engine/rules.py` konfigurierbar ablegen statt hart anzunehmen.

---

4. Datensammlung (implementiert – bei Bedarf erweitern)

Zentrale Erkenntnis aus der Projektentwicklung: Hochrangige Spieler bilden einen kleinen, dicht vernetzten Pool – Matchmaking wirft sie immer wieder zusammen. Ein Snowball-Crawl sättigt diesen Pool vergleichsweise schnell, danach läuft die Queue leer. Das ist kein Sackgassenproblem, sondern ein Strategiewechsel: Statt einmalig breit zu crawlen, muss derselbe bekannte Pool danach wiederholt abgeklappert werden, um neue Matches einzusammeln – weil `battlelog` immer nur ein rollierendes Fenster zeigt, nicht die Historie.

- Erster Lauf: breite Discovery über Trophäen-Rangliste als Seed (`seed_tags()`).
- Folgeläufe: `--reseed-from-rank` re-enqueued bekannte Spieler mit Rang ≥ Schwelle mit hoher Priorität – gedacht für periodisches Polling per Cronjob, nicht als Dauerprozess.
- Dedup über Battle-Hash (battleTime + sortierte Teilnehmer-Tags), da die API keine Battle-ID liefert. Bei mehrfacher Sichtung desselben Kampfs (aus verschiedenen Battlelogs) wird der höher beobachtete `rank`-Wert übernommen.

---

5. Rang-Filterung & Gewichtung

Zielbereich: Legendary III bis Pro, über `min_rank` (numerisch, siehe Abschnitt 3) gefiltert. Lineares Gewicht oberhalb der Schwelle über `rank_weight()`, Schrittgrösse `weight_step` tunbar. Dieses Gewicht wird beim Modelltraining (Abschnitt 8) als `sample_weight` verwendet – höherrangige Spiele fliessen stärker gewichtet ein, werden aber nicht per hartem Cutoff ausgeschlossen.

---

6. Bekannte Dateneinschränkungen – bei Abschnitt 8 und 9 zwingend zu respektieren

Bei ca. 70 Brawlern gibt es C(70,3) ≈ 55'000 mögliche 3er-Team-Kompositionen, also grob 3 Milliarden mögliche volle Team-vs-Team-Matchups (multipliziert mit der Map-Anzahl). Die exakte volle Draft-Situation – alle 6 Bans und alle Picks identisch zu einer bereits gesehenen Situation – wird praktisch nie zweimal vorkommen, selbst bei hunderttausenden gesammelten Spielen.

Das ist kein Grund, an der Datenmenge zu zweifeln, sondern die Begründung für den Modellierungsansatz: kleinere Bausteine – ein einzelner Brawler, ein Teamkollegen-Paar, eine Gegnerschaft, Teil-Kombinationen wie „2 identische Bans + 3 identische Picks“ – kommen tausendfach vor, auch wenn die exakte Gesamtsituation neu ist. Das Modell muss aus diesen häufig beobachteten Bausteinen komponieren können. Konkrete Implikation fürs Feature-Engineering in Abschnitt 8: **kein** One-Hot-Encoding der kompletten 6-Brawler-Situation als atomares Feature, sondern additive Features auf Brawler- und Paar-Ebene.

Weiterer Punkt: eine naive „Winrate mit Brawler X im Team“-Zählung verwechselt echte Synergie mit X's allgemeiner Stärke (Confounding) – ein starker Brawler hebt jedes Team, unabhängig von Synergie. Das löst ein regularisiertes Modell (Abschnitt 8), keine Rohzählung (Abschnitt 7).

Letzter Punkt: der Sammlungs-Durchsatz ist durch das Rate-Limit und das 25-Kämpfe-Fenster begrenzt, nicht dadurch, wie viele Ranked-Spiele weltweit existieren.

---

7. Explorative Statistik (implementiert)

`stats.py` liefert `brawler_winrate()`, `synergy_matrix()` und `matchup_matrix()`, jeweils mit `min_samples`-Schwelle als Rauschfilter. Diese Funktionen sind zum Debuggen und als Sanity-Check gedacht – **nicht** als produktive Empfehlungsquelle fürs Draft-Tool. Das übernimmt das Modell in Abschnitt 8.

---

8. Kernstück: Win-Probability-Modell (noch zu bauen)

Input: Map + Team A (bis zu 3 Brawler) + Team B (bis zu 3 Brawler), muss auch mit unvollständigen Teams während des laufenden Drafts funktionieren (noch nicht alle Slots gefüllt). Output: geschätzte Siegwahrscheinlichkeit von Team A.

Feature-Design (siehe Abschnitt 6):

- Pro Brawler ein Indikator-Feature (je Team-Seite getrennt), zusätzlich explizite Paar-Interaktionsterme (Teamkollegen-Paare, Gegner-Paare) statt reinem Encoding der vollen Kombination – damit das Modell aus häufig gesehenen Bausteinen auf seltene Gesamtsituationen generalisiert.
- Fehlende Slots (während des Drafts) explizit als „leer“/Padding kodieren, oder das Modell direkt auf Partial-State-Eingaben trainieren.

Modellwahl:

Gradient Boosting (XGBoost/LightGBM) *oder* regularisierte logistische Regression mit expliziten Interaktionstermen – beide Ansätze evaluieren. Gradient Boosting findet nützliche Interaktionen tendenziell automatisch über Splits, explizite Paar-Features sind trotzdem als Zusatz-Features sinnvoll.

Training & Validierung:

- Training mit `sample_weight` = Ausgabe von `rank_weight()` (Abschnitt 5).
- Split nach **Zeit**, nicht zufällig – die Meta ändert sich laufend durch Balance-Patches; ein zufälliger Split schleust Informationen aus der Zukunft in die Vergangenheit ein und überschätzt die Performance.
- Kalibrierung reporten (vorhergesagte vs. tatsächliche Winrate in Bins), nicht nur Accuracy – für eine Empfehlungs-Engine ist eine gut kalibrierte Wahrscheinlichkeit wichtiger als reine Trefferquote.
- Bei erkannter Fehlkalibrierung (Bins weichen systematisch ab): nachgeschalteten Kalibrierungsschritt anwenden, z.B. Platt-Scaling oder Isotonic Regression, statt die rohe Modellausgabe direkt als Wahrscheinlichkeit auszugeben.

---

9. Kernstück: Draft-Such-Engine (noch zu bauen – das eigentliche „AI bot“)

Implementiert die echte Ranked-Draftregel-Sequenz aus Abschnitt 3 (3 Bans pro Team, danach alternierende Picks). Exakte Reihenfolge/Gleichzeitigkeit der Bans vorher am Live-Client verifizieren, dann konfigurierbar in

`rules.py` ablegen – nicht hart annehmen.

Ban-Phase:

Nicht einfach global höchste Winrate bannen. Für jeden Kandidaten-Ban berechnen, wie stark er das bestmögliche Endergebnis des Gegners über den Rest des Baums senkt (Denial-Wert) – siehe Abschnitt 1a.

Pick-Phase:

Kein reiner Greedy-Pick (nur eigene unmittelbare Siegchance maximieren). Für jeden Kandidaten-Pick den bestmöglichen gegnerischen Antwort-Pick simulieren; den Kandidaten bevorzugen, der die stärkste gegnerische Antwort am meisten schwächt – Minimax mit dem Modell aus Abschnitt 8 als Bewertungsfunktion an den Blattknoten. Siehe Abschnitt 1b.

Suchtiefe & Performance:

Mindestens über alle verbleibenden Picks/Bans im Baum suchen. Bei ca. 70 Kandidaten pro Zug ist Full-Width-Suche teuer – Kandidatenliste vor der Baum-Expansion auf Top-N (z.B. 15–20) nach Modell-Score einschränken, plus Alpha-Beta-Pruning.

Ausgabe:

Jede Empfehlung zeigt mindestens: empfohlener Brawler, geschätzte resultierende Siegwahrscheinlichkeit, sowie optional die 2–3 nächstbesten Alternativen zum Vergleich.

Struktur als Pseudocode (illustrativ, keine finale Implementierung):

```
function best_action(state, depth):
  if state.is_complete() or depth == 0:
    return model.predict(state.map, state.team_a, state.team_b)

  candidates = top_n_by_heuristic(state.available_brawlers, n=15)
  values = []
  for action in candidates:
    child = state.apply(action)
    v = best_action(child, depth - 1)
    values.append((action, v))

  if state.turn == "us":
    return max(values, key=value)  # eigener Zug: maximieren
  else:
    return min(values, key=value)  # Gegner spielt optimal gegen uns
```

Ban-Bewertung folgt demselben Baum, nur mit anderem Wert pro Kandidat: Differenz im Baumwert *mit* vs. *ohne* den Brawler im verfügbaren Pool, statt der Brawler-Stärke selbst (siehe „Ban-Phase“ oben).

---

10. Backend – FastAPI (noch zu bauen)

- `/recommend`: Map, aktuelle Bans (beide Seiten), aktuelle Picks (beide Seiten), aktuelle Phase/wer am Zug → sortierte Liste von Empfehlungen mit Win%-Delta.
- Modell und Brawlerliste einmal beim Start laden, nicht pro Request.
- Liefert zusätzlich das Frontend aus Abschnitt 11 als statische Dateien aus (`StaticFiles`-Mount) – ein Service, kein separates Frontend-Deployment (siehe Abschnitt 0).

---

11. Frontend – Web-UI (noch zu bauen)

Umsetzung als Vanilla HTML/CSS/JS ohne Build-Step, siehe Abschnitt 0 für die verbindliche Stack-Entscheidung. Zielgerät: Computer/grosser Bildschirm (kein Mobile-First nötig), primärer Anwendungsfall ist ein zweiter Bildschirm während eines laufenden, zeitlich begrenzten Live-Drafts. **Kernanforderung: so wenig Klicks und so wenig Blicksprünge wie möglich** – jede Interaktion, die vermeidbar ist, kostet in einem echten Draft echte Zeit.

Zentrale UX-Entscheidungen:

- **State-Machine statt manueller Buchführung.** Die App leitet Phase (Ban/Pick) und wer am Zug ist selbst aus der Anzahl bisheriger Aktionen ab (Regeln aus Abschnitt 3/9). Der Nutzer klickt nur den Brawler – nicht zusätzlich, ob das ein Ban oder Pick für welches Team ist.
- **Dichtes Icon-Grid statt Liste/Dropdown** für alle Brawler – Brawler werden optisch erkannt, nicht gelesen. Ein Suchfeld (autofokussiert) filtert per Texteingabe als schneller Fallback. Bereits gebannte/gepickte Brawler werden ausgegraut und sind nicht mehr klickbar – der verfügbare Pool schrumpft sichtbar mit dem Draft-Fortschritt.
- **Empfehlung direkt im Grid sichtbar**: Top-Kandidaten bekommen einen farbigen Rahmen direkt an ihrem Icon (kein separates Nachschlagen nötig), zusätzlich eine kompakte Liste mit 2–3 Alternativen und Win%-Werten daneben.
- **Ein-Tasten-Bestätigung**: `Enter` übernimmt sofort die Top-Empfehlung in den aktuellen Slot, ohne Maus. Das ist der wichtigste Geschwindigkeits-Hebel im ganzen UI.
- **Timeline-Leiste** oberhalb des Grids: kompakter, horizontaler Verlauf aller bisherigen Bans/Picks in tatsächlicher Reihenfolge, aktueller Slot hervorgehoben – schneller Sichtcheck, ob der bisherige Draft-Stand korrekt erfasst wurde.
- **Undo jederzeit sichtbar** (Button und `Esc`) – Fehlklicks unter Zeitdruck sind unvermeidlich, die Korrektur muss so schnell gehen wie der Fehler selbst.
- **Kein Submit-Button, keine Ladeseite.** Jeder Klick/jede Tastatureingabe löst sofort einen neuen Call an `/recommend` (Abschnitt 10) aus; die Empfehlung aktualisiert sich live.
- **Dunkles Theme, hoher Kontrast, minimale Animation** – für schnelles, wiederholtes Ablesen unter Zeitdruck, nicht für visuelle Verspieltheit.
- Map-Auswahl zu Beginn: durchsuchbares Icon-Raster wie der Brawler-Grid, kein langes Dropdown.

Grobes Layout (Wireframe, illustrativ):

```
+-------------------------------------------------------------------+
| [Neuer Draft]   Shooting Star - Ban-Phase 4/6, Team B dran [Undo]|
+-------------------------------------------------------------------+
| Ban Ban Ban | Ban Ban Ban | Pick Pick Pick | Pick Pick Pick      |
| [P] [B] [.] | [E] [.] [.] |  .    .    .   |  .    .    .       |
+-------------------------------------+-------------------------+
| /Suche...                           | EMPFEHLUNG                |
|                                   |  #1 Amber      72%      |
| [dichtes Icon-Grid, verfügbare     |     (Enter = übernehmen) |
|  Brawler, Top-Picks goldener     |  #2 Bibi       68%      |
|  Rahmen, gebannte/gepickte       |  #3 Chester   65%      |
|  ausgegraut]                        |                           |
+-------------------------------------------------------------------+
| Enter: Top-Pick übernehmen | Esc: Rückgängig | /: Suche        |
+-------------------------------------------------------------------+
```

[P]/[B]/[E] in der Timeline stehen beispielhaft für bereits eingetragene Brawler (Kürzel/Icon), [.] für noch offene Slots. Team A obere Zeile, Team B untere Zeile innerhalb der jeweiligen Phase.

---

12. Deployment

- Ziel: Railway, Hobby-Plan (~5 USD/Monat inkl. Nutzungsguthaben) – Account existiert bereits.
- SQLite-Datenbank braucht ein persistentes Railway-Volume, sonst bei jedem Redeploy verloren.
- Crawler läuft **nicht** als Dauerprozess: initialer breiter Lauf, danach als Cron-Service mit `--reseed-from-rank` in konfigurierbarem Intervall.

Railway Cron Schedule (konkreter Mechanismus):

- Service → Settings → Feld „Cron Schedule“ → Crontab-Ausdruck (z.B. `0 */2 * * *` für alle 2h). Railway führt dann den Start-Command des Service nach diesem Schedule aus.
- Der Service muss sich nach getaner Arbeit selbst beenden (unser Crawler tut das bereits) – Railway killt keine noch laufende Ausführung automatisch. Läuft ein Durchlauf länger als das Cron-Intervall, wird der nächste Trigger einfach übersprungen, nicht nachgeholt. Intervall entsprechend grosszügig zur tatsächlichen Laufzeit wählen.
- Empfohlener Ablauf: ersten breiten Discovery-Lauf einmal manuell anstossen (ohne Cron Schedule), danach Start-Command auf die Reseed-Variante umstellen und Cron Schedule aktivieren.

---

13. Realistische Erwartungen (bitte beim Bauen im Hinterkopf behalten)

Es wird kein Orakel. In Solo-Ranked spielt man mit zufälligen Mitspielern, nicht einem eingespielten Team – das Matchergebnis hängt stark vom individuellen Skill/der Mechanik der 6 Beteiligten ab, nicht nur vom Draft. Selbst ein optimal trainiertes Modell hat dadurch eine reale Vorhersage-Obergrenze, die auch mehr Daten nicht auflösen. Zielsetzung ist eine **statistische Kante** gegenüber Bauchgefühl (vergleichbar oder besser als die Referenz-Tools aus Abschnitt 1) – kein Anspruch auf nahezu perfekte Vorhersagegenauigkeit.

Realistischer Zeitrahmen: Datensammlung, Modell, Such-Engine und Web-Tool bis zu einem tatsächlich nutzbaren Stand sind eher ein Projekt über mehrere Wochen als ein einmaliger Durchlauf – die Checkliste in Abschnitt 14 wird sich beim Bauen wahrscheinlich mehrfach verlängern, das ist normal und kein Zeichen, dass etwas falsch läuft.

---

14. Offene Punkte – Checkliste vor Fertigstellung

- [ ] Ranked `type`/`mode`-Wert bestätigt (Abschnitt 3)
- [ ] `min_rank` für Legendary III numerisch bestätigt (Abschnitt 3)
- [ ] Exakte Ban/Pick-Sequenz am Live-Client verifiziert (Abschnitt 3, 9)
- [ ] `weight_step` und `min_samples` anhand der tatsächlich gesammelten Datenmenge kalibriert (Abschnitt 5, 7)
- [ ] Modell-Kalibrierung mit zeitbasiertem Split geprüft, nicht nur Accuracy (Abschnitt 8)
- [ ] Denial-Logik (Ban-Phase) und Lookahead-Logik (Pick-Phase) je an mind. einem konkreten Beispiel-Draft manuell nachvollzogen (Abschnitt 9)

Ende der Spezifikation. Bei Unklarheiten: lieber im bestehenden Repo/den bereits getroffenen Entscheidungen nachschauen oder eine sinnvolle, begründete Annahme treffen und dokumentieren, als zu raten – siehe Abschnitt 3 für das Muster „verifiziert vs. noch zu verifizieren“.